

Keras 3

An *unfair advantage*
for ML devs



Introduction to Keras



Keras Mission

What is Keras
Why you should use it



API Overview

Basic workflow
Making new layers
Custom loss
Custom train step/loop



Why Multibackend

Origin story
Benchmarking
JAX & PyTorch support



around the world

450,000 package downloads
per day on average



Netflix



Spotify



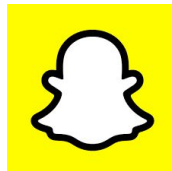
Youtube



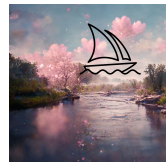
GrubHub



Twitter/X



Snap

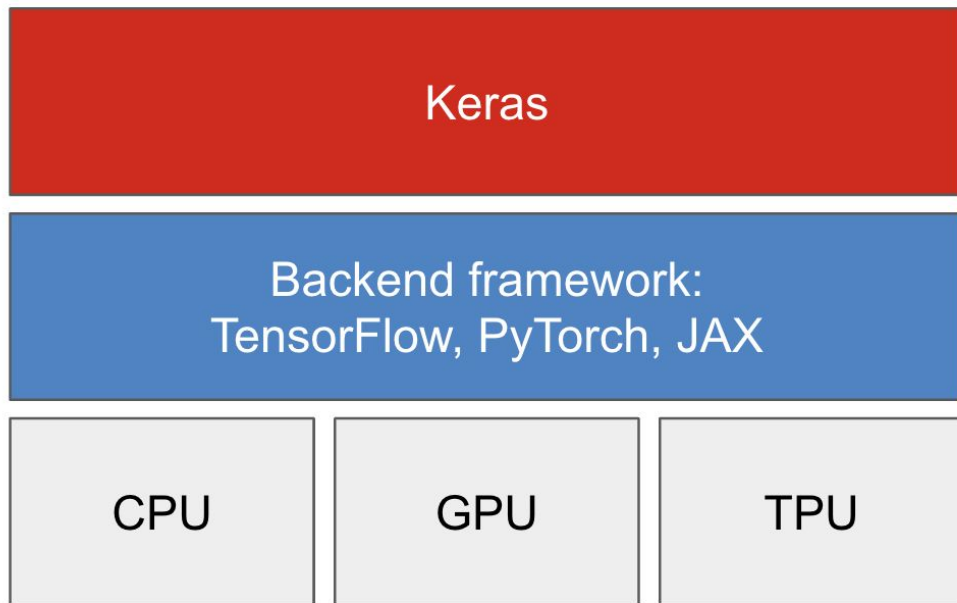


Midjourney



WAYMO
Waymo

Keras 3: a multi-backend deep learning library



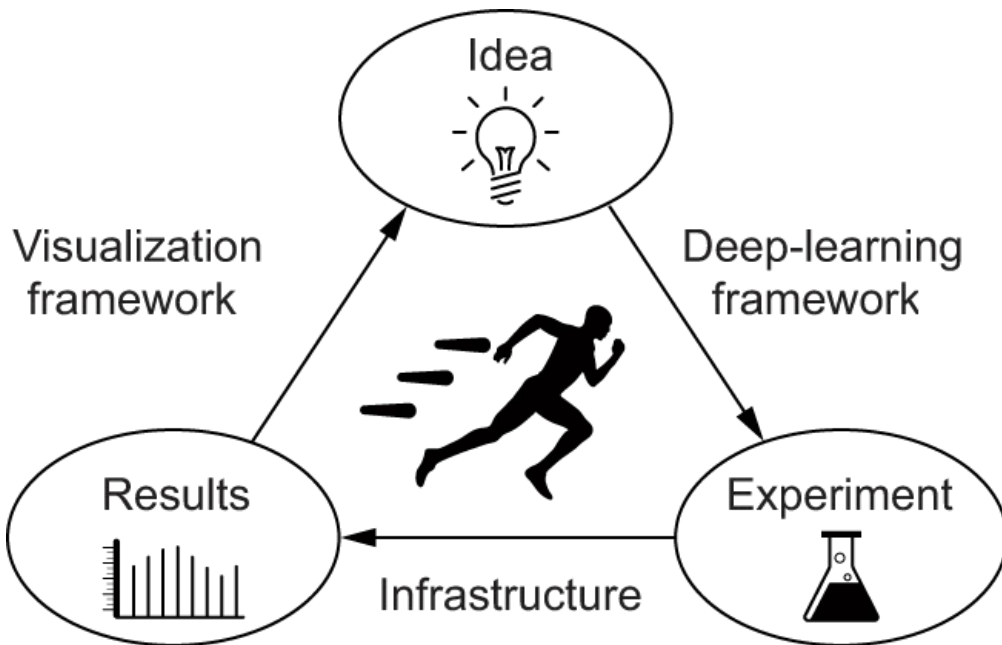
Deep learning development:
layers, models, optimizers, losses,
metrics, training loops...

Tensor manipulation infrastructure:
tensors, variables, automatic
differentiation, distribution...

Hardware: execution

The loop of progress

build great products,
win competitions, get papers published



Who uses Keras?

Basic users



As little code as possible

Batteries included

Best-practices baked-in by default

Engineers

Flexible model architecture

Flexible training logic

Scale and speed

Deployment-readiness

Researchers



Don't want a framework, just a toolbox

Low-level control of every detail

Simple, consistent programming mental models

The Keras API philosophy:

Progressive disclosure of complexity



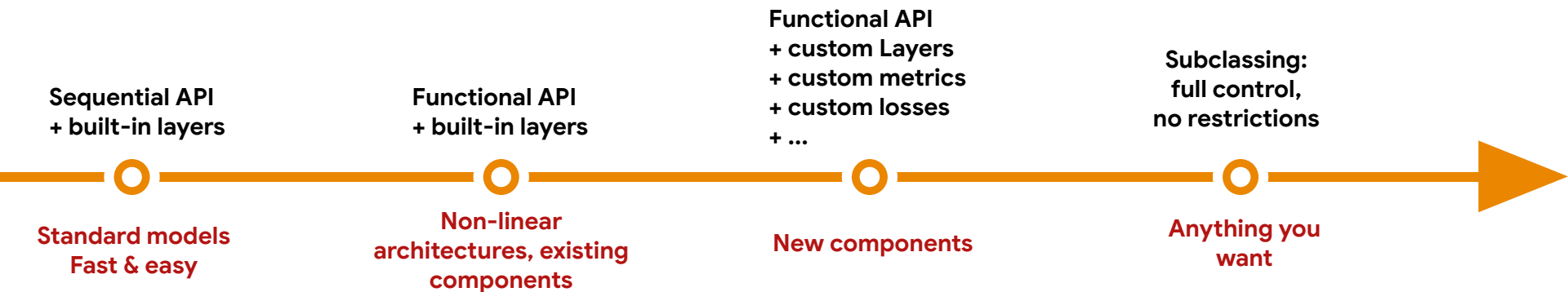
API Overview

Building your model

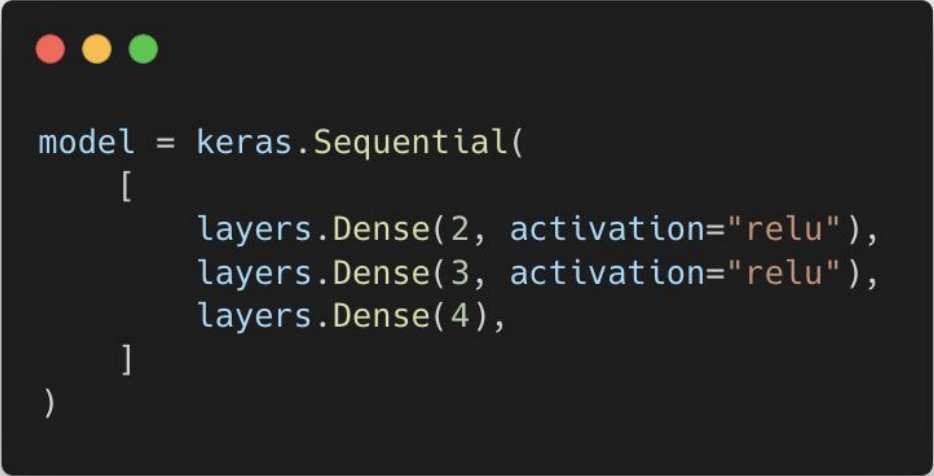


Model building

From **simple** to **arbitrarily flexible**



The Sequential API



```
model = keras.Sequential(  
    [  
        layers.Dense(2, activation="relu"),  
        layers.Dense(3, activation="relu"),  
        layers.Dense(4),  
    ]  
)
```



```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(3, 2)	8
layer2 (Dense)	(3, 3)	9
layer3 (Dense)	(3, 4)	16

Total params: 33 (132.00 B)

Trainable params: 33 (132.00 B)

Non-trainable params: 0 (0.00 B)

The Functional API

- Multi-input, multi-output models
- Checks for input compatibility at construction time
 - Saves a lot of debugging time!
- Graph structure easily inspectable

```
# Embed each word in the title into a 64-dimensional vector
title_input = keras.Input(shape=(None,), name="title")
title_features = layers.Embedding(num_words, 64)(title_input)

# Embed each word in the text into a 64-dimensional vector
body_input = keras.Input(shape=(None,), name="body")
body_features = layers.Embedding(num_words, 64)(body_input)

# Reduce sequence of embedded words in the title into a single 128-dimensional vector
title_features = TransformerEncoder(128, num_heads=2)(title_features)
title_features = layers.GlobalAveragePooling1D()(title_features)

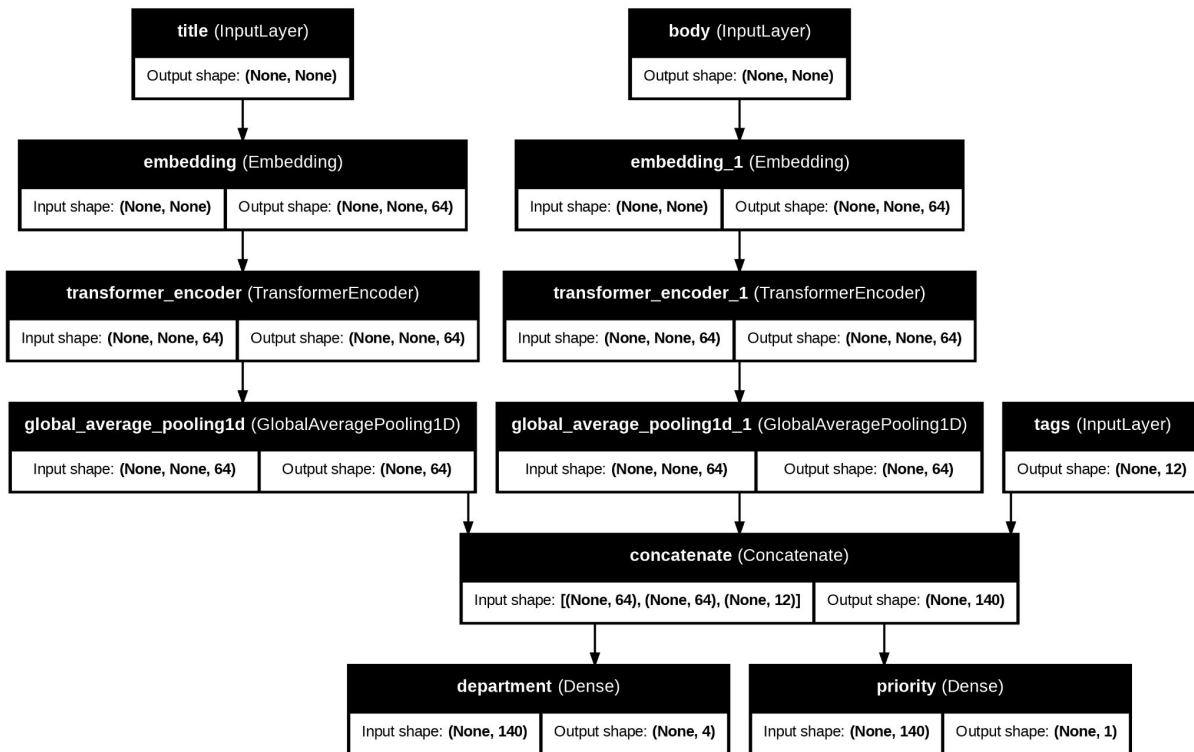
# Reduce sequence of embedded words in the body into a single 32-dimensional vector
body_features = TransformerEncoder(32, num_heads=2)(body_features)
body_features = layers.GlobalAveragePooling1D()(body_features)

# Add 3rd input; merge all features via concatenation, add a logistic regression
tags_input = keras.Input(shape=(num_tags,), name="tags")
x = layers.concatenate([title_features, body_features, tags_input])
priority_pred = layers.Dense(1, name="priority")(x)

# Stick a department classifier on top of the features
department_pred = layers.Dense(num_departments, name="department")(x)

# Instantiate an end-to-end model predicting both priority and department
model = keras.Model(
    inputs=[title_input, body_input, tags_input],
    outputs={"priority": priority_pred, "department": department_pred},
)
```

```
keras.utils.plot_model(model, "multi_input_and_output_model.png", show_shapes=True)
```





Creating custom layers

- Create weights in `__init__()` or `build()`
- Specify computation in `call()`

```
class Linear(keras.layers.Layer):
    def __init__(self, units=32):
        super().__init__()
        self.units = units

    def build(self, input_shape):
        self.w = self.add_weight(
            shape=(input_shape[-1], self.units),
            initializer="random_normal",
            trainable=True,
        )
        self.b = self.add_weight(
            shape=(self.units,), initializer="random_normal", trainable=True
        )

    def call(self, inputs):
        return ops.matmul(inputs, self.w) + self.b
```

The keras.ops API

- **A superset of NumPy**
 - Except it's cross-framework, GPU accelerated, TPU accelerated, ready for multi-device distribution
- **245 ops** that all run with JAX, PyTorch, TF, NumPy
- Includes 145 NumPy ops and 100 ops for NNs
 - e.g. dot_product_attention with FlashAttention, CTC, separable convolutions...

```
import numpy as np

def my_np_function(x, y):
    z = np.matmul(x, y)
    z = np.stack((z, x), axis=0)
    return np.maximum(np.exp(y), 0.)
```



```
from keras import ops

def my_keras_function(x, y):
    z = ops.matmul(x, y)
    z = ops.stack((z, x), axis=0)
    return ops.maximum(ops.exp(y), 0.)
```

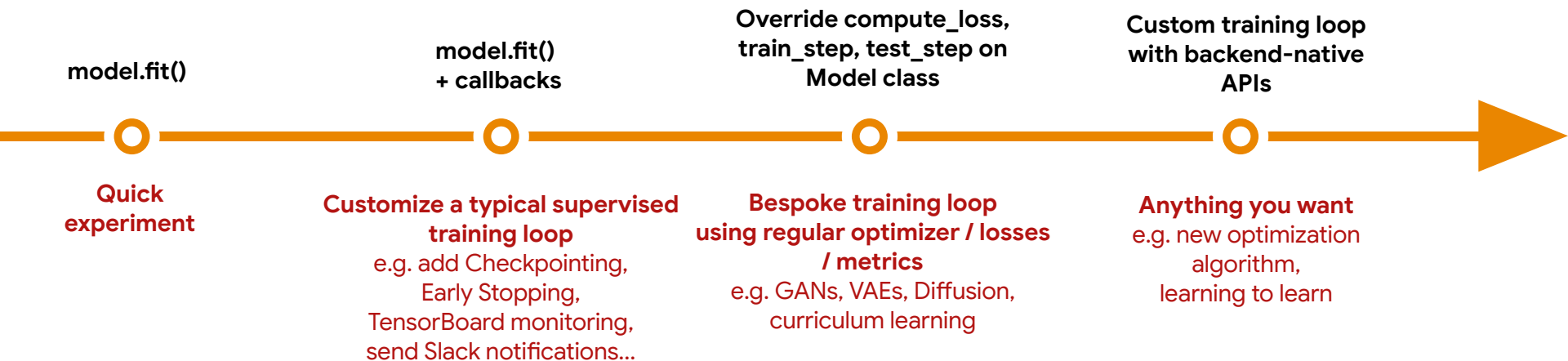
API Overview

Training your model

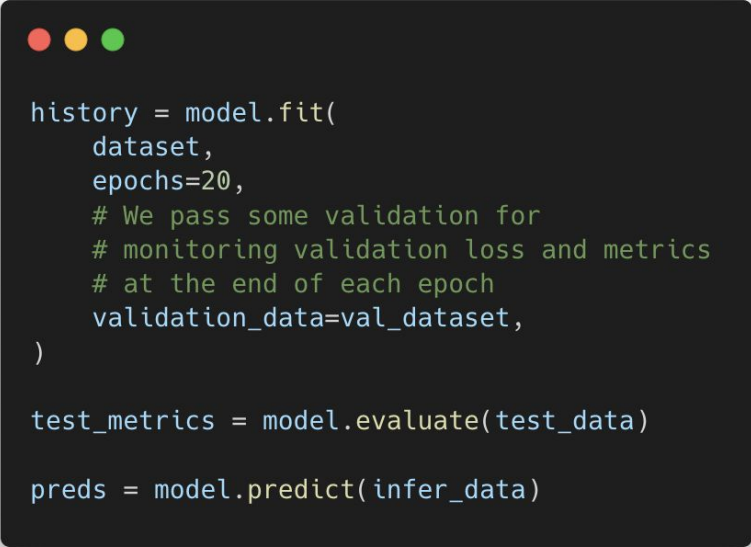


Model training

From **simple** to **arbitrarily flexible**

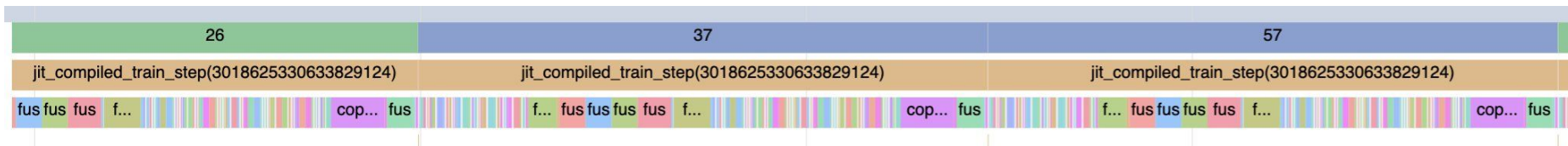


The built-in fit, evaluate, predict loops



```
history = model.fit(  
    dataset,  
    epochs=20,  
    # We pass some validation for  
    # monitoring validation loss and metrics  
    # at the end of each epoch  
    validation_data=val_dataset,  
)  
  
test_metrics = model.evaluate(test_data)  
  
preds = model.predict(infer_data)
```

Gotta go fast



... Keras does a lot of optimizations under the hood!

Customizing fit/predict/evaluate with callbacks

```
callbacks = [  
    # Stop training when `val_loss` is no longer improving  
    keras.callbacks.EarlyStopping(  
        monitor="val_loss",  
        min_delta=1e-2,  
        patience=2,  
        verbose=1,  
    ),  
    # Save checkpoints at each epoch, overwrite with the best one  
    keras.callbacks.ModelCheckpoint(  
        filepath="mymodel.keras",  
        monitor="val_loss",  
        save_best_only=True,  
    ),  
]  
  
model.fit(  
    dataset,  
    epochs=20,  
    callbacks=callbacks,  
    validation_data=val_dataset,  
)
```



Creating custom callbacks

```
class LossHistory(keras.callbacks.Callback):  
    def on_train_begin(self, logs):  
        self.per_batch_losses = []  
  
    def on_batch_end(self, batch, logs):  
        self.per_batch_losses.append(logs.get("loss"))
```

Creating custom metrics or losses

```
def my_metric_fn(y_true, y_pred):  
    squared_difference = ops.square(y_true - y_pred)  
    return ops.mean(squared_difference, axis=-1)  
  
model.compile(optimizer='adam', loss='mean_squared_error', metrics=[my_metric_fn])
```

Simple functions

...



```
class BinaryTruePositives(keras.metrics.Metric):

    def __init__(self, name='binary_true_positives', **kwargs):
        super().__init__(name=name, **kwargs)
        self.true_positives = self.add_weight(name='tp', initializer='zeros')

    def update_state(self, y_true, y_pred, sample_weight=None):
        y_true = ops.cast(y_true, "bool")
        y_pred = ops.cast(y_pred, "bool")

        values = ops.logical_and(ops.equal(y_true, True), ops.equal(y_pred, True))
        values = ops.cast(values, self.dtype)
        if sample_weight is not None:
            sample_weight = ops.cast(sample_weight, self.dtype)
            values = values * sample_weight
        self.true_positives.assign_add(ops.sum(values))

    def result(self):
        return self.true_positives

    def reset_state(self):
        self.true_positives.assign(0)
```

Or arbitrarily
complex stateful
metrics

Creating custom optimizers – literally everything can be customized



```
class SGD(Optimizer):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.momentum = 0.9

    def build(self, variables):
        super().build(variables)
        self.momentums = []
        for variable in variables:
            self.momentums.append(
                self.add_variable_from_reference(
                    reference_variable=variable, name="momentum"
                )
            )

    def update_step(self, gradient, variable, learning_rate):
        learning_rate = ops.cast(learning_rate, variable.dtype)
        gradient = ops.cast(gradient, variable.dtype)
        m = self.momentums[self._get_variable_index(variable)]
        self.assign(
            m,
            ops.subtract(
                ops.multiply(m, ops.cast(self.momentum, variable.dtype)),
                ops.multiply(gradient, learning_rate),
            ),
        )
        self.assign_add(variable, m)
```

Customizing the training loop with compute_loss: VAE example



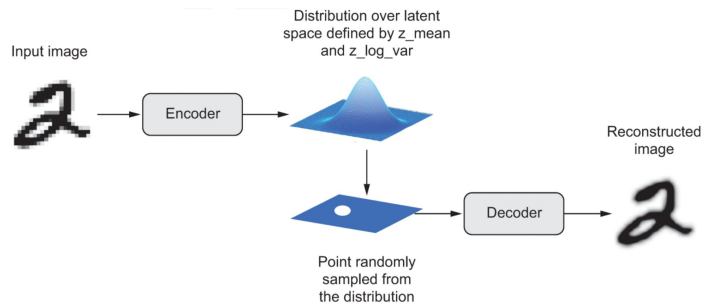
```
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.sampler = Sampler()
        self.reconstruction_loss_tracker = keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    def call(self, inputs):
        return self.encoder(inputs)

    def compute_loss(self, x, y, y_pred, sample_weight=None, training=True):
        original = x
        z_mean, z_log_var = y_pred
        reconstruction = self.decoder(self.sampler(z_mean, z_log_var))

        reconstruction_loss = ops.mean(
            ops.sum(
                keras.losses.binary_crossentropy(x, reconstruction),
                axis=(1, 2)
            )
        )
        kl_loss = -0.5 * (1 + z_log_var - ops.square(z_mean) - ops.exp(z_log_var))
        total_loss = reconstruction_loss + ops.mean(kl_loss)

        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)
        return total_loss
```



Customizing the training loop with compute_loss: Diffusion example

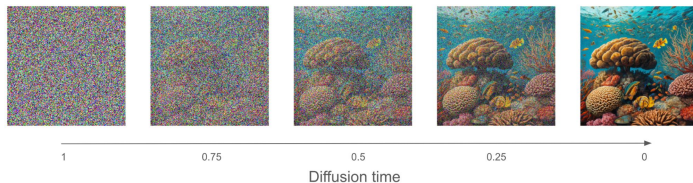


```
class DiffusionModel(keras.Model):
    def __init__(self, image_size, widths, block_depth, **kwargs):
        super().__init__(**kwargs)
        self.image_size = image_size
        self.denoising_model = get_model(image_size, widths, block_depth)
        self.seed_generator = keras.random.SeedGenerator()
        self.loss = keras.losses.MeanAbsoluteError()
        self.normalizer = keras.layers.Normalization()

    def denoise(self, noisy_images, noise_rates, signal_rates):
        pred_noise_masks = self.denoising_model([noisy_images, noise_rates])
        pred_images = (noisy_images - noise_rates * pred_noises) / signal_rates
        return pred_images, pred_noise_masks

    def call(self, images):
        images = self.normalizer(images)
        noise_masks = keras.random.normal(
            (batch_size, self.image_size, self.image_size, 3), seed=self.seed_generator)
        diffusion_times = keras.random.uniform(
            (batch_size, 1, 1, 1), minval=0.0, maxval=1.0, seed=self.seed_generator)
        noise_rates, signal_rates = diffusion_schedule(diffusion_times)
        noisy_images = signal_rates * images + noise_rates * noise_masks
        pred_images, pred_noise_masks = self.denoise(noisy_images, noise_rates, signal_rates)
        return pred_images, pred_noise_masks, noise_masks

    def compute_loss(self, x, y, y_pred, sample_weight=None, training=True):
        _, pred_noise_masks, noise_masks = y_pred
        return self.loss(noise_masks, pred_noise_masks)
```





Researchers

**Data
scientists**

Engineers

The Multi-Backend Advantage



Keras over the years

March 2015

First version of Keras



November 2023

Keras 3 is released with support for multiple backends



November 2019

Keras integrated into TensorFlow 2.0



October 2024

Keras Hub 🎉



Creating backend-native custom layers: JAX, PyTorch, TF

```
import torch

class Linear(keras.layers.Layer):
    ...

    def call(self, inputs):
        return torch.matmul(inputs, self.w) + self.b
```

```
import jax

class Linear(keras.layers.Layer):
    ...

    def call(self, inputs):
        return jax.numpy.matmul(inputs, self.w) + self.b
```

```
import tensorflow as tf

class Linear(keras.layers.Layer):
    ...

    def call(self, inputs):
        return tf.matmul(inputs, self.w) + self.b
```



Works with backend-native ecosystems

Common question 🤔

*Can I use Keras APIs
along with native
JAX/TF/PyTorch APIs?*

Yep.

- JAX: Keras components can be used side by side with Flax Modules
 - You can turn any Flax Module into a Keras layer in one layer via the `FlaxLayer` class
- PyTorch: Keras components can be used side by side with PyTorch Modules
 - Keras layers and models ARE PyTorch Modules!
- TensorFlow: yep, that one too

Writing a backend-native train_step: PyTorch

- Call `zero_grad()`
- Call `loss.backward()`
- Retrieve gradients via `v.value.grad`

```
class CustomModel(keras.Model):
    def train_step(self, data):
        # Unpack the data. Its structure depends on your model and
        # on what you pass to `fit()`.
        x, y = data

        # Call torch.nn.Module.zero_grad() to clear the leftover gradients
        # for the weights from the previous train step.
        self.zero_grad()

        # Compute loss
        y_pred = self(x, training=True) # Forward pass
        loss = self.compute_loss(y=y, y_pred=y_pred)

        # Call torch.Tensor.backward() on the loss to compute gradients
        # for the weights.
        loss.backward()

        trainable_weights = [v for v in self.trainable_weights]
        gradients = [v.value.grad for v in trainable_weights]

        # Update weights
        with torch.no_grad():
            self.optimizer.apply(gradients, trainable_weights)

        # Update metrics (includes the metric that tracks the loss)
        for metric in self.metrics:
            if metric.name == "loss":
                metric.update_state(loss)
            else:
                metric.update_state(y, y_pred)

        # Return a dict mapping metric names to current value
        # Note that it will include the loss (tracked in self.metrics).
        return {m.name: m.result() for m in self.metrics}
```

Writing a backend-native train_step: TensorFlow

- Open GradientTape, compute loss
- Retrieve gradients via tape.gradients

```
class CustomModel(keras.Model):
    def train_step(self, data):
        # Unpack the data. Its structure depends on your model and
        # on what you pass to `fit()`.
        x, y = data

        with tf.GradientTape() as tape:
            y_pred = self(x, training=True) # Forward pass
            # Compute the loss value
            # (the loss function is configured in `compile()`)
            loss = self.compute_loss(y=y, y_pred=y_pred)

        # Compute gradients
        trainable_vars = self.trainable_variables
        gradients = tape.gradient(loss, trainable_vars)

        # Update weights
        self.optimizer.apply(gradients, trainable_vars)

        # Update metrics (includes the metric that tracks the loss)
        for metric in self.metrics:
            if metric.name == "loss":
                metric.update_state(loss)
            else:
                metric.update_state(y, y_pred)

        # Return a dict mapping metric names to current value
        return {m.name: m.result() for m in self.metrics}
```

Writing a backend-native training loop: PyTorch

- Works with PyTorch native optimizers and losses
 - Can use Keras ones too ofc
- Call `zero_grad()`
- Call `loss.backward()`
- Call `optimizer.step()`

```
# Instantiate a Keras model
model = get_model()

# Instantiate a torch optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# Instantiate a torch loss function
loss_fn = torch.nn.CrossEntropyLoss()

epochs = 3
for epoch in range(epochs):
    for step, (inputs, targets) in enumerate(train_dataloader):
        # Forward pass
        logits = model(inputs)
        loss = loss_fn(logits, targets)

        # Backward pass
        model.zero_grad()
        loss.backward()

        # Optimizer variable updates
        optimizer.step()

        # Log every 100 batches.
        if step % 100 == 0:
            print(
                f"Training loss (for 1 batch) at step {step}: "
                f"{loss.detach().numpy():.4f}"
            )
            print(f"Seen so far: {(step + 1) * batch_size} samples")
```

Writing a backend-native training loop: TensorFlow

- Open GradientTape, compute loss
- Retrieve gradients via tape.gradient
- Apply optimizer

```
# Instantiate a Keras model.
model = get_model()

# Instantiate an optimizer.
optimizer = keras.optimizers.Adam(learning_rate=1e-3)

# Instantiate a loss function.
loss_fn = keras.losses.SparseCategoricalCrossentropy(from_logits=True)

epochs = 3
for epoch in range(epochs):
    print(f"\nStart of epoch {epoch}")

    # Iterate over the batches of the dataset.
    for step, (x_batch_train, y_batch_train) in enumerate(train_dataset):
        # Open a GradientTape to record the operations run
        # during the forward pass, which enables auto-differentiation.
        with tf.GradientTape() as tape:
            logits = model(x_batch_train, training=True)
            loss_value = loss_fn(y_batch_train, logits)

        # Use the gradient tape to automatically retrieve
        # the gradients of the trainable variables with respect to the loss.
        grads = tape.gradient(loss_value, model.trainable_weights)

        # Run one step of gradient descent by updating
        # the value of the variables to minimize the loss.
        optimizer.apply(grads, model.trainable_weights)

    # Log every 100 batches.
    if step % 100 == 0:
        print(
            f"Training loss (for 1 batch) at step {step}: "
            f"{float(loss_value):.4f}"
        )
    print(f"Seen so far: {(step + 1) * batch_size} samples")
```

Writing a backend-native training loop: JAX

- Works with JAX native `jax.jit`, `jax.value_and_grad`
 - Why wrap them? You should be using JAX.
- Make loss/update function
- Make gradient function
- Make jitted `train_step`
- Iterate statelessly via
 - `loss, state = train_step(state, data)`

```
def compute_loss_and_updates(trainable_variables, non_trainable_variables, x, y):
    y_pred, non_trainable_variables = model.stateless_call(
        trainable_variables, non_trainable_variables, x
    )
    loss = loss_fn(y, y_pred)
    return loss, non_trainable_variables

grad_fn = jax.value_and_grad(compute_loss_and_updates, has_aux=True)

@jax.jit
def train_step(state, data):
    trainable_variables, non_trainable_variables, optimizer_variables = state
    x, y = data
    (loss, non_trainable_variables), grads = grad_fn(
        trainable_variables, non_trainable_variables, x, y
    )
    trainable_variables, optimizer_variables = optimizer.stateless_apply(
        optimizer_variables, grads, trainable_variables
    )
    # Return updated state
    return loss, (
        trainable_variables,
        non_trainable_variables,
        optimizer_variables,
    )

# Build optimizer variables.
optimizer.build(model.trainable_variables)

trainable_variables = model.trainable_variables
non_trainable_variables = model.non_trainable_variables
optimizer_variables = optimizer.variables
state = trainable_variables, non_trainable_variables, optimizer_variables

# Training loop
for step, data in enumerate(train_dataset):
    data = (data[0].numpy(), data[1].numpy())
    loss, state = train_step(state, data)
    # Log every 100 batches.
    if step % 100 == 0:
        print(f"Training loss (for 1 batch) at step {step}: {float(loss):.4f}")
        print(f"Seen so far: {(step + 1) * batch_size} samples")
```



Common question 🤔
**Which backend
should I choose?**

XLA is fast 🧠 !

And JAX is the best
way to leverage it
💪

See the benchmark
numbers ➡



	Batch size	Keras 2 (TensorFlow)	Keras 3 (TensorFlow)	Keras 3 (JAX)	Keras 3 (PyTorch) (eager)	Keras 3 (best)
SegmentAnything (fit)	1	386.93	355.25	361.69	1,388.87	355.25
SegmentAnything (predict)	4	1,859.27	438.50	376.34	1,720.96	376.34
Stable Diffusion (fit)	8	1,023.21	392.24	391.21	823.44	391.21
Stable Diffusion (predict)	13	649.71	616.04	627.27	1,337.17	616.04
BERT (fit)	32	486.00	214.49	222.37	808.68	214.49
BERT (predict)	256	470.12	466.01	418.72	1,865.98	418.72

Source: https://keras.io/getting_started/benchmarks/



Performance stacking

From slowest to fastest (rough order):

1. PyTorch eager 
2. TF graph mode 
3. PyTorch compiled 
4. TF (XLA compiled) 
5. JAX (also XLA compiled) 

KerasHub

Pretrained Model Hub for
Keras 3 🔥





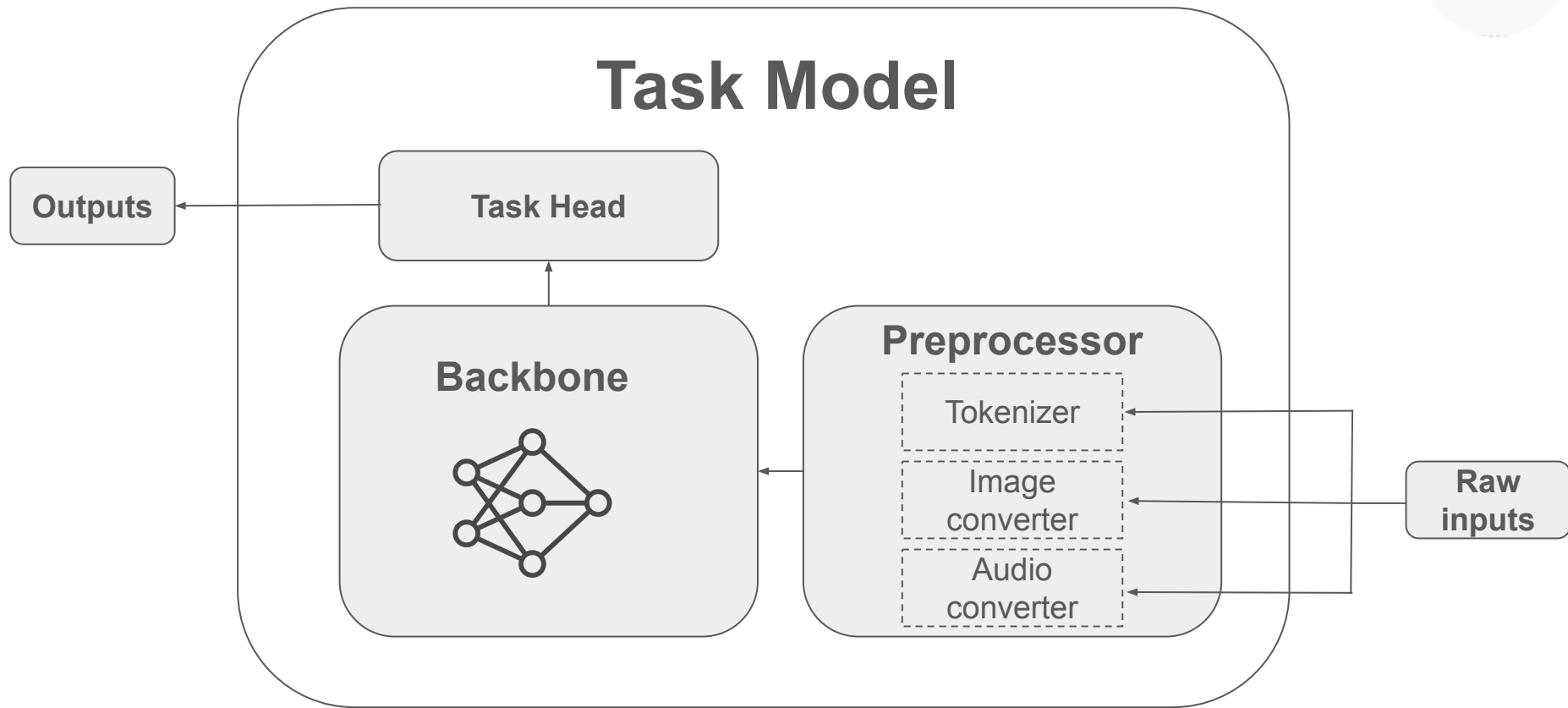
Get started with Keras Hub

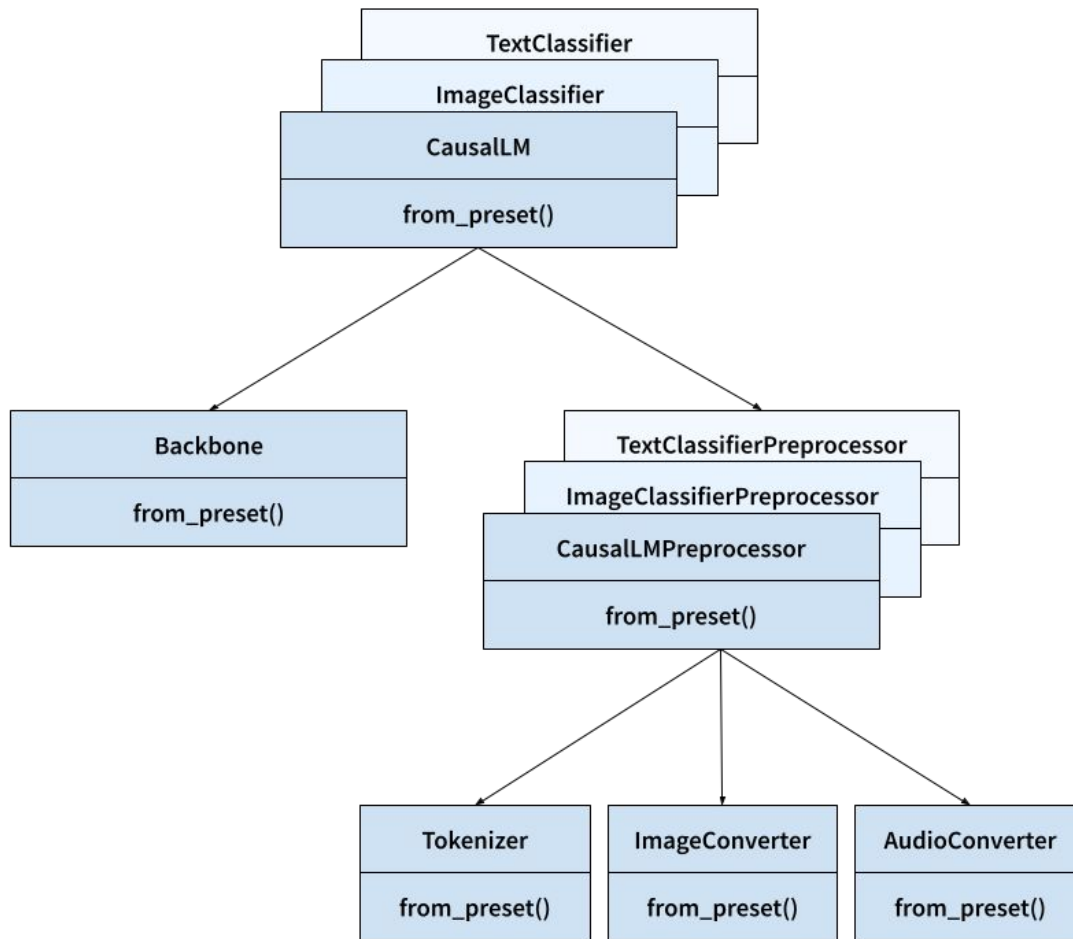
```
import os

# Define the Keras 3 backend you want to use
os.environ["KERAS_BACKEND"] = "jax"

# Import Keras 3 and KerasHub modules
import keras
import keras_hub

# Load Gemma 2 2B preset from Kaggle models
gemma_lm = keras_hub.models.GemmaCausalLM.from_preset("gemma_2b_en")
```







Customize your model

```
tokenizer = keras_hub.models.GemmaTokenizer(  
    proto="proto.spm",  
)  
preprocessor = keras_hub.models.GemmaCausalLMPreprocessor(  
    tokenizer=tokenizer,  
    sequence_length=128,  
)  
backbone = keras_hub.models.GemmaBackbone(  
    vocabulary_size=30552,  
    num_layers=4,  
    num_heads=4,  
    hidden_dim=256,  
    intermediate_dim=512,  
    max_sequence_length=128,  
)  
gemma_lm = keras_hub.models.GemmaCausalLM(  
    backbone=backbone,  
    preprocessor=preprocessor,  
)  
gemma_lm.fit(x=features, batch_size=2)
```



Customize your model

```
preprocessor = keras_hub.models.GemmaCausalLMPreprocessor.from_preset(  
    "gemma2_instruct_2b_en",  
    dtype="bfloat16",  
    sequence_length=64, # Override the sequence_length arg  
)  
backbone = keras_hub.models.GemmaBackbone.from_preset(  
    "gemma2_instruct_2b_en")  
gemma_lm = keras_hub.models.GemmaCausalLM(  
    backbone=backbone,  
    preprocessor=preprocessor,  
)
```

KerasHub
on
HuggingFace 🤗
kaggle



Loading model weights from HF and Kaggle

Load preset
from Kaggle
URL

```
keras_hub.models.CausalLM.from_preset("kaggle://my_kaggle_username/gemma-pirate/keras/gemma-pirate-instruct-7b")
```

Load weights
from
HuggingFace
URL

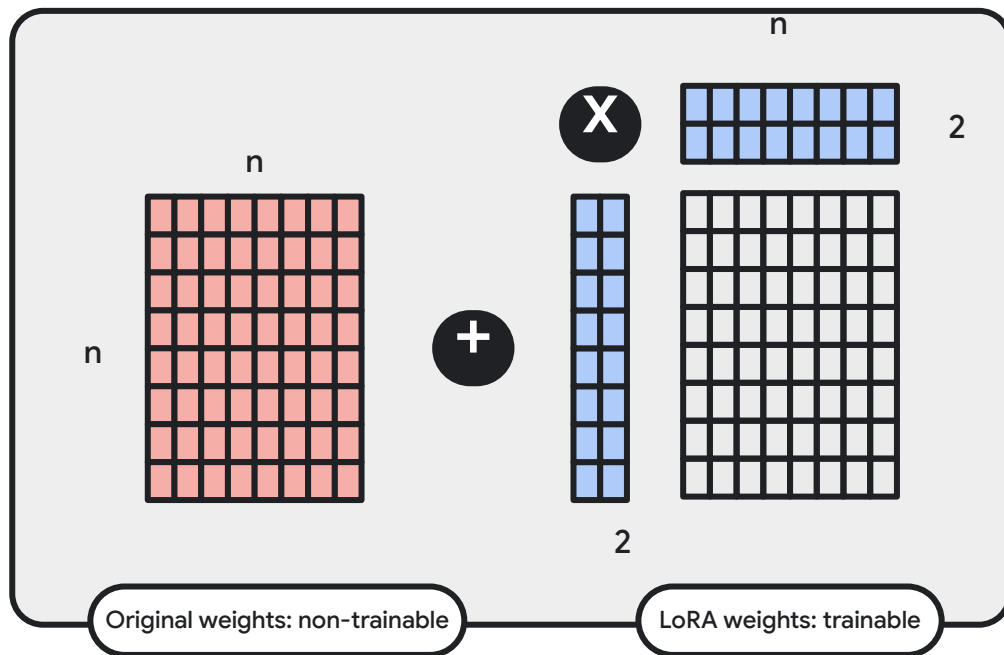
```
keras_hub.models.CausalLM.from_preset("hf://NousResearch/Hermes-2-Pro-Llama-3-8B")
```

Fine-tuning with KerasHub

Follow along at
go/keras-ai-seminar-colab



LORA Fine Tuning



LORA Fine Tuning

Enable LoRA

```
# load model ...  
  
gemma.backbone.enable_lora(rank=2)  
  
# fine-tune ...
```

Save / Load
LoRA weights

```
gemma.backbone.save_lora_weights("lora.h5")  
gemma.backbone.load_lora_weights("lora.h5")
```

QLORA Fine Tuning

Quantize the
weights

```
# load model ...  
gemma_lm.quantize("int8")
```

Enable LoRA

```
gemma_lm.backbone.enable_lora(rank=4)  
# fine-tune ...
```

Try it!!

ViTDet
GPTNeoX RetinaNet
ResNet EfficientNet VAE
Whisper PaliGemma MobileNet
T5 FNet OPT
SAM Electra CLIP
DebertaV3 Bert DistilBert
DenseNet Albert
xlnet MiT Bloom Bart Gemma GPT2
CSPDarknet Llama3 DeepLabv3
Mistral
StableDiffusion3
VGG Falcon Phi3 XLMRoberta
Roberta

The background is a solid red color. It features several concentric white circles. The innermost circle is solid, while the others are dashed. There are also several small white stars scattered across the background.

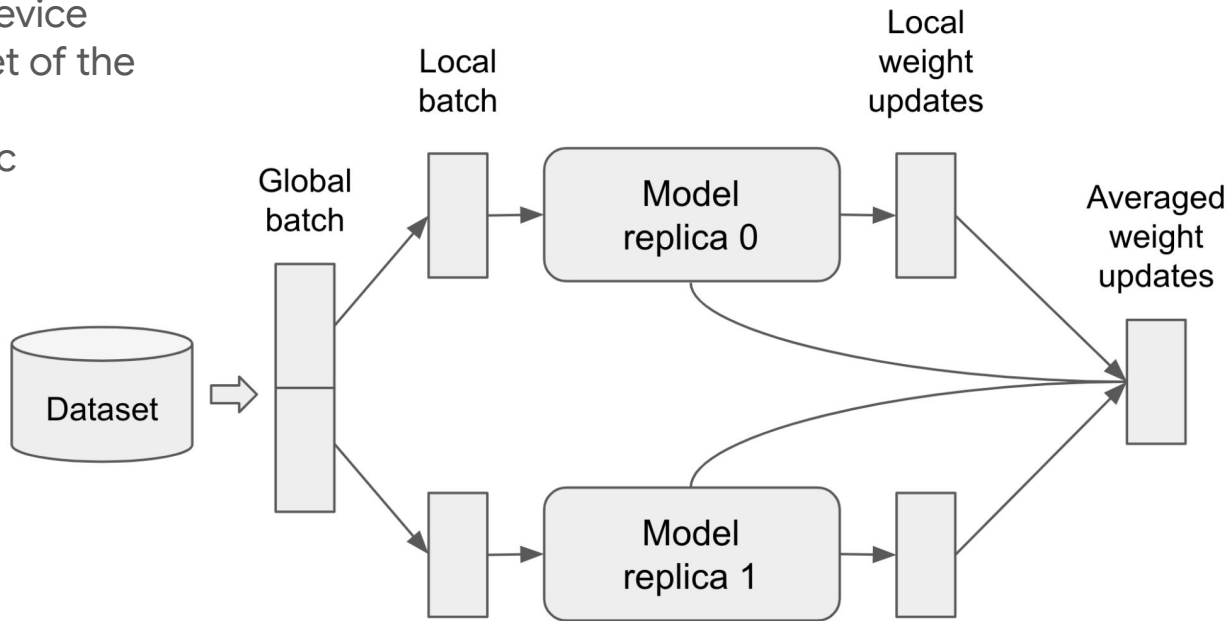
Deep Dive

Scaling Keras with JAX



Understanding data-parallel distribution

- M model replicas
- Each one on its own device
- Each one sees a subset of the global batch
- All replicas kept in sync





Data parallel distribution in one line

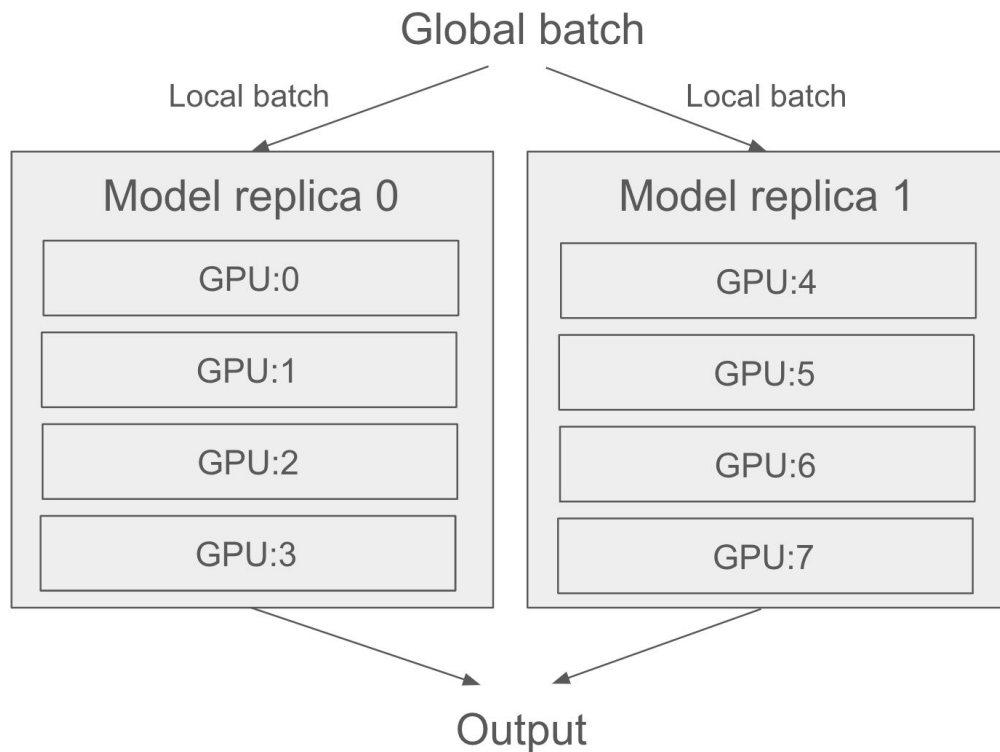
```
keras.distribution.set_distribution(  
    keras.distribution.DataParallel()  
)
```

```
keras.distribution.set_distribution(  
    keras.distribution.DataParallel(devices=["gpu:0", "gpu:1"])  
)
```



Understanding model-parallel distribution

- M model replicas
- Each one split across D devices
- Total of $M * D$ devices
- All replicas kept in sync



Model parallel distribution with LayoutMap

```
# Assume 16 devices...
devices = keras.distribution.list_devices()
device_mesh = keras.distribution.DeviceMesh((2, 8), ["batch", "model"], devices)
layout_map = keras.distribution.LayoutMap(device_mesh)

# Partitioning for embeddings (regex)
layout_map["token_embedding/embeddings"] = (None, "model")

# Partitioning (regex) for attention layer weights
layout_map["decoder_block.*attention.*(query|key|value).*kernel"] = (None, "model", None)
layout_map["decoder_block.*attention_output.*kernel"] = (None, None, "model")
layout_map["decoder_block.*ffw_gating.*kernel"] = ("model", None)
layout_map["decoder_block.*ffw_linear.*kernel"] = (None, "model")

keras.distribution.set_distribution(
    keras.distribution.ModelParallel(
        device_mesh,
        Layout_map,
        batch_dim_name="batch",
    )
)

model = get_model()
model.fit(...)
metrics = model.evaluate(...)
preds = model.predict(...)
```



```
# Assume 16 devices...
devices = keras.distribution.list_devices()
device_mesh = keras.distribution.DeviceMesh((2, 8), ["batch", "model"], devices))
layout_map = keras.distribution.LayoutMap(device_mesh)

# Partitioning for embeddings (regex)
layout_map["token_embedding/embeddings"] = (None, "model")

# Partitioning (regex) for attention layer weights
layout_map["decoder_block.*attention.*(query|key|value).*kernel"] = (None,
"model", None)
layout_map["decoder_block.*attention_output.*kernel"] = (None, None, "model")
layout_map["decoder_block.*ffw_gating.*kernel"] = ("model", None)
layout_map["decoder_block.*ffw_linear.*kernel"] = (None, "model")

keras.distribution.set_distribution(
    keras.distribution.ModelParallel(
        device_mesh,
        Layout_map,
        batch_dim_name="batch",
    )
)

model = get_model()
model.fit(...)
metrics = model.evaluate(...)
preds = model.predict(...)
```



Learn more:

- <https://keras.io>
- <https://github.com/keras-team/keras>

For customer inquiries or collaboration opportunities with Keras 3 and the Keras ecosystem, please reach out to keras-team@google.com